CS612 · ADVANCED ALGORITHMS · MSC · FALL 2026

# Lecture 11
# String Algorithms & Multi-Pattern Matching

Deterministic automata vs. probabilistic fingerprints, and what changes when there are ten thousand patterns.

Saturday 5 December 2026 · 3 hours

PS3 due today · PS4 released next week

# Motivating problem

A security team scans $10^9$ characters of logs per day for any of $5{,}000$ indicator strings (lengths 6–40).

- One pattern at a time with a good single-pattern algorithm: $5000$ passes over $10^9$ characters.
- Compare hashes of windows with a hash set of pattern hashes? Fast, but **what if two different strings have equal hashes?**

**EQUAL HASHES ≠ EQUAL STRINGS**

By pigeonhole, a hash with $p$ values has collisions among $p + 1$ strings. Collisions are not only theoretical:

- Two different PDF files with the same SHA-1 hash were published in 2017 (the "SHAttered" attack).
- Polynomial hashes modulo $2^{64}$ with a fixed base are broken by Thue–Morse strings, a well-known competitive-programming trap.

BLOCK A · 00:10–01:00

# One pattern

Baseline · rolling hashes · Rabin–Karp · KMP

# The baseline: naive matching

Try every shift $s = 0..n - m$ and compare characters until the first mismatch.

- **Worst case $\Theta(nm)$:** text $a^n$, pattern $a^{m-1}b$. Every shift compares $m$ characters.
- **Random text** over an alphabet of size $\sigma$: the expected comparisons per shift are $\leq \frac{1}{1-1/\sigma} \leq 2$. Expected $O(n)$ for any fixed pattern.
- It needs no preprocessing, no memory, and has a tight inner loop. Real libraries often use naive search or variants with skips (Boyer–Moore–Horspool).

**WHY A "BAD" ALGORITHM IS A GOOD BASELINE**

It is correct by inspection, so it is the **oracle** for differential testing, and on natural text it is hard to beat. The value of the others lies in their **guarantees**: worst-case linear (KMP), and the ability to compare many things at once (hashing, automata).

# Polynomial rolling hashes

For a window $x = x_0 x_1 \cdots x_{m-1}$ (characters as integers $< p$), a prime $p$ and a base $b \in \{0, \ldots, p-1\}$:

$$h_b(x) = \Big( \sum_{i=0}^{m-1} x_i \, b^{m-1-i} \Big) \bmod p.$$

**Rolling:** sliding the window from $T[s..s+m-1]$ to $T[s+1..s+m]$:

$$h \leftarrow \big( (h - T[s]\, b^{m-1}) \cdot b + T[s+m] \big) \bmod p \qquad O(1) \text{ per shift.}$$

**COLLISION BOUND (IDEA)**

Fix $x \neq y$ of length $m$ **before** choosing $b$ uniformly at random. Then $h_b(x) - h_b(y)$ is a nonzero polynomial in $b$ of degree $\leq m-1$ over the field $\mathbb{Z}_p$, so it has at most $m-1$ roots: $\Pr_b[\text{collision}] \leq \frac{m-1}{p}$.

**CONDITIONS THAT MATTER**

- $b$ must be random and chosen after the strings are fixed (an oblivious adversary).
- $p$ must be prime (a field). Working modulo $2^{64}$ is not a field.
- The bound is per comparison: use a union bound over all windows.

# Rabin–Karp: Monte Carlo or Las Vegas

```
RABIN-KARP(T, P)
  choose b uniformly at random      ▷ after T and P are fixed
  hp = h(P) ; ht = h(T[0..m−1])
  for s = 0 to n − m
    if ht == hp
      if T[s..s+m−1] == P           ▷ verification
        report s
    roll ht to the next window
```

| | Report on hash match | Verify each hash match |
|---|---|---|
| Type | Monte Carlo (false positives) | Las Vegas (always correct) |
| Time | $O(n+m)$ worst case | expected $O(n+m+\text{occ}\cdot m)$ + false hits × $m$ |
| Worst case | $O(n+m)$ | $O(nm)$ (many true matches, e.g. $a^n$ / $a^m$) |

**LEGACY CORRECTION**

The old code reported a match on hash equality and claimed "$O(N+M)$". That is a Monte Carlo algorithm presented as exact. If a false match is unacceptable, verify.

# KMP: the prefix function

A **border** of a string is a proper prefix that is also a suffix.

$\pi[i]$ = the length of the longest border of $P[0..i]$.

| $i$ | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| $P$ | a | a | b | a | a | a | b |
| $\pi$ | 0 | 1 | 0 | 1 | 2 | 2 | 3 |

**Key fact:** the borders of $P[0..i]$ are exactly $\pi[i]$, $\pi[\pi[i]-1]$, … (the **border chain**).

```
KMP-MATCH(T, P, π)
  q = 0                          ▷ length of the current match
  for each character c of T
    while q > 0 and P[q] ≠ c
      q = π[q − 1]               ▷ fall back to the next border
    if P[q] == c: q = q + 1
    if q == m
      report a match ; q = π[m − 1]
```

**INVARIANT**

After reading $T[0..i]$, $q$ = the length of the longest prefix of $P$ that is a suffix of $T[0..i]$.

# KMP matching is linear: a potential argument

**CLAIM**

KMP-MATCH makes $O(n)$ character comparisons on a text of length $n$, whatever the text.

**PROOF**

Let $\Phi = q \geq 0$, with $\Phi = 0$ initially. Each text character increases $q$ by at most 1 (the single `q = q + 1`). Each iteration of the while-loop decreases $q$ strictly ($\pi[q-1] < q$), and so does the reset after a match. The total decrease cannot exceed the total increase, which is $\leq n$. So there are $\leq n$ while-iterations in total, and the total work is $O(n)$. (Amortized: $\leq 2$ comparisons per text character.)

■

**DETERMINISTIC GUARANTEE**

$O(n+m)$ worst case for *every* input: no probability, no adversary model. It is a streaming algorithm (reads each character once) with $O(m)$ memory.

Computing $\pi$ itself in $O(m)$ uses the same kind of argument with a different potential. (Proof: PS4.)

# Exercise A: single-pattern reasoning

1. Compute $\pi$ for $P =$ `abcabcabd` and list the border chain of $P[0..7]$.
2. Give a text on which KMP performs a long run of fallbacks on a **single** character. How long can it be for $|P| = m$?
3. Rabin–Karp with $p = 2^{31} - 1$, $m = 50$, text $n = 10^9$: bound the probability of any false hit. Is that acceptable for an index used by a search engine? What would you change?
4. Show that computing $\pi$ of the single string $P\,\$\,T$ (with a separator $\$$ occurring in neither) finds all occurrences of $P$ in $T$. Why is the separator needed?

Pairs · 8 min

BLOCK B · 01:10–02:00

# Many patterns

Tries · Aho–Corasick · (optional) suffix arrays

# Tries: the starting point

- A rooted tree whose edges are labelled with characters. Each pattern is a root-to-node path, and terminal nodes are marked.
- Insert/search a string of length $\ell$: $O(\ell)$ steps, independent of the number of stored strings.
- Size: at most $M + 1$ nodes for patterns of total length $M$, with shared prefixes stored once.

| Children stored as | Step | Memory per node |
|---|---|---|
| array of size $\sigma$ | $O(1)$ | $\sigma$ pointers |
| hash map | $O(1)$ expected | $O(\text{children})$, with overhead |
| sorted list / BST | $O(\log \sigma)$ | $O(\text{children})$ |

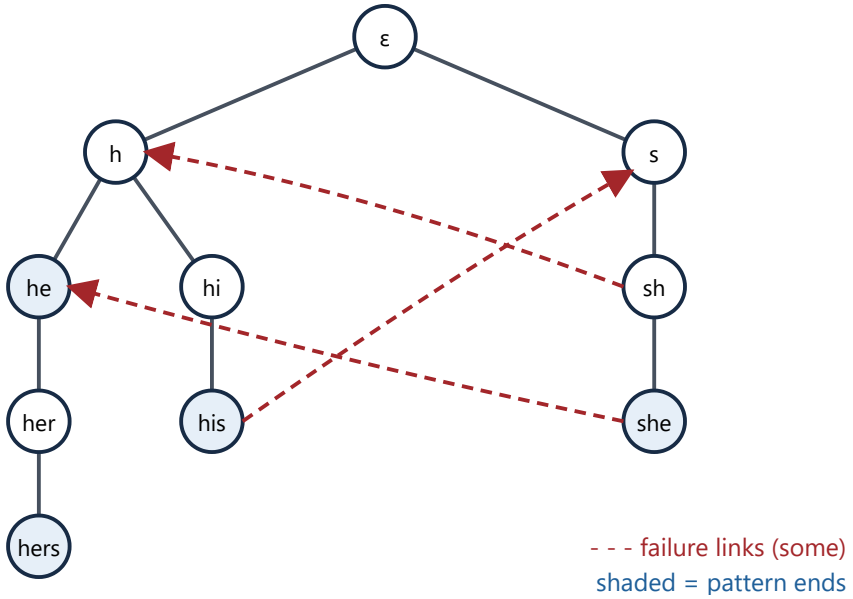
**ASK**

To find occurrences of *any* of $k$ patterns in a text, walk the trie from every text position. What does that cost? What is wasted when a walk fails?

# Aho–Corasick (1975): KMP on a trie

- **Failure link** $f(v)$: the node for the **longest proper suffix** of $v$'s string that is also in the trie. (For one pattern, this is exactly $\pi$.)
- **Output (dictionary) link:** the nearest node on the failure chain that ends a pattern. It reports every pattern that ends here.
- **Matching:** read the text; follow a trie edge if one exists, otherwise follow failure links. Report outputs at each position.

Example patterns: he, she, his, hers.



# Aho–Corasick: construction, correctness, cost

```
BUILD-FAILURE(trie)
  f(root) = root ; children of root: f = root
  BFS over nodes v (by depth)
    for each child u = child(v, c)
      x = f(v)
      while x ≠ root and child(x, c) undefined
        x = f(x)
      f(u) = child(x, c) if defined (and ≠ u) else root
      out(u) = f(u) if f(u) ends a pattern else out(f(u))
```

**Why BFS:** $f(u)$ is shorter than $u$, so it is already final when $u$ is processed.

**INVARIANT (MATCHING)**

After reading $T[0..i]$, the current node is the **longest suffix** of $T[0..i]$ that is a trie node. Every pattern ending at $i$ is a suffix of it, and all of them lie on its output chain.

**COMPLEXITY**

Build $O(M\sigma)$ with a full goto table ($O(M)$ with maps); match $O(n + \mathrm{occ})$ with output links. **Independent of** $k$, apart from $M$.

$\mathrm{occ}$ can be huge: patterns $a, aa, \ldots, a^k$ in text $a^n$ give $\Theta(nk)$ matches. Output-sensitive, not linear.

# Optional: indexing the text instead (suffix arrays)

When the **text** is fixed and patterns arrive later (a search index), preprocess the text instead of the patterns.

- **Suffix array** SA: the starting positions of all suffixes, in sorted order. Build in $O(n \log n)$ (prefix doubling) or $O(n)$ (SA-IS).
- **Search** for $P$: binary search over SA: $O(m \log n)$ ( $O(m + \log n)$ with LCP information).
- **LCP array**: the longest common prefix of adjacent sorted suffixes, in $O(n)$ (Kasai).
- Example: the longest repeated substring = the maximum of LCP.

| Preprocess | Structure | Query |
|---|---|---|
| one pattern | $\pi$ (KMP) | $O(n)$ per text |
| many patterns | Aho–Corasick | $O(n + \text{occ})$ per text |
| the text | suffix array + LCP | $O(m \log n + \text{occ})$ per pattern |
| nothing | naive / Rabin–Karp | $O(nm)$ / $O(n + m)$ expected |

# Exercise B: Aho–Corasick practice

1. Build the trie and failure links for patterns {`a`, `ab`, `bab`, `bc`, `bca`, `c`, `caa`}. Which nodes have non-trivial output links?
2. Trace the automaton on the text `abccab` and list every reported (position, pattern).
3. Why can't we report only the pattern that ends at the current node? Give an example that shows why output links are needed.

Groups · 10 min

BLOCK C · 02:10–02:55

# Evidence and selection

Guarantees vs. constants · one pattern vs. many

# Experiments: single and multiple patterns

| Text $n = 3 \times 10^7$, $m = 1000$ | naive | KMP | RK | RK false hits |
|---|---|---|---|---|
| random, $\sigma = 26$ | 31 | 25 | 275 | 0 |
| random, $\sigma = 2$ | 138 | 131 | 266 | 0 |
| $a^n$ / $a^{m-1}b$ | **10,516** | **50** | **269** | **0** |

| $k$ patterns (length 8), $n = 10^7$ | $k$ ×KMP | Aho–Corasick | AC states |
|---|---|---|---|
| 1 | 8 | 22 | 9 |
| 100 | 818 | 23 | 718 |
| 1,000 | 8,249 | 44 | 6,509 |
| **10,000** | – | **96** | **58,216** |

ms; RK: random base, $p = 2^{61} - 1$, every hash hit verified; `Student_Resources/strings_bench.cpp`, g++ 15.2 -O2.

**INTERPRET · 6 MIN**

1. Naive beats RK on random text. Why, and what is RK's constant made of?
2. What guarantee does the adversarial row illustrate?
3. AC's time grows from 22 to 96 ms while $k$ grows 10,000×. What explains the growth that does occur?
4. For $k = 1$, KMP beats AC. Why?

# Exercise C: choose and justify

1. **Plagiarism screening:** report whether two 200-page documents share any passage of $\geq 50$ consecutive words.
2. **A genome browser:** a fixed 3 GB reference genome; millions of short read queries (length 100) per day, exact matches only.
3. **An intrusion-detection sensor:** 20,000 byte signatures; a 10 Gbit/s packet stream; a worst-case per-byte budget.
4. **A text editor's "find":** one pattern typed by a user, in a 5 MB file.

> **GROUPS · 12 MIN**
>
> For each: the algorithm/index, its guarantee (deterministic? expected?), memory, and the one assumption that would change your choice.

# Synthesis

1. Naive is the correct baseline and oracle: expected fast on natural text, $\Theta(nm)$ in the worst case.
2. Polynomial hashing with a **random base modulo a prime**: collision $\leq (m-1)/p$ per comparison; union bound over windows. **Equal hashes never prove equal strings.** Verify for Las Vegas.
3. KMP: the border chain + the invariant; linear by a potential argument; deterministic and streaming.
4. Aho–Corasick: KMP on a trie; $O(n + M + \mathrm{occ})$, independent of $k$. Mind $\mathrm{occ}$ and memory.
5. Choose by what is fixed: the pattern (KMP), a pattern set (AC), or the text (suffix arrays).

**NEXT WEEK · L12 (SAT 12 DEC)**

Network flow: residual graphs, augmenting paths, max-flow/min-cut, Edmonds–Karp and Dinic, bipartite matching, and the skill of **modelling problems as flow**.

**Read:** KT 7.1–7.6; CLRS 4e Ch. 24. **PS4** released next week; **M5** (preliminary results) due Sat 12 Dec.